

예; $\sigma_{age>25}(\text{users})$ = 나이 25세 이상 사용자

예; $\pi_{\text{user_id}, \text{name}}(\text{users})$ = id와 이름만

$$\sigma_{\text{age}>25}(E\mu(R)) \equiv E\mu(\sigma_{\text{age}>25}(R))$$

그런데 ****비용이 다르다!****

/~개성문자-에 /~군각갯자-에 /~x~각-0 F

- └ Consume: 테이블의 행(row)을 읽을 때
| 해당 컨텍스트 데이터도 함께 임베딩
|
- └ Exchange: 임베딩 결과를 다른 연산자에 전달
| (예; 조인 연산자가 임베딩 벡터를 직접 사용)
|
- └ Cache: 동일 데이터에 대한 반복 임베딩 방지
| 예; 같은 user_image를 여러 쿼리에서 사용할 때
| 첫 번째 쿼리에서 계산한 임베딩을 재사용

S4e 경교과교 /~게거강개용y개0 F

$$\sigma_p(E\mu(R)) \equiv E\mu(\sigma_p(R))$$

쿼리: 임베딩한 후, 나이 > 25인 사용자만 필터

[계획 A] 임베딩 먼저 (비효율)

- 1. 전체 1,000,000명 사용자
- 2. 각각 ResNet 실행 (임베딩)
 - $1,000,000 \times 100\text{ms} = 100\text{K초}$
- 3. 나이 필터 적용
 - 결과 500,000명 (50%)

비용: 100K초 + IO + 필터링

[계획 B] 필터 먼저 (효율적)

1. 나이 > 25 조건으로 필터
→ 결과 500,000명
2. 500,000명만 임베딩
→ $500,000 \times 100ms = 50K초$

비용: 필터링 + 50K초 + IO

$$4x \text{ 변태도출량} \times \text{비분류량} \div 5$$

즉, 프로젝션할 컬럼을 임베딩 이전에 미리 선택할 수 있다.

예시:
최종 결과에서 user_id와 embedding만 필요하다면,
user_name, user_email은 임베딩 전에 버려도 된다.
(임베딩 모델의 입력에는 user_image만 필요하므로)

RAP a

쿼리: 1,000,000명 사용자 중
- 나이 > 25 (선택도 50%)
- 임베딩 후 유사 이미지 검색

동치 규칙에 의해 가능한 실행 계획들:

[계획 1] 필터 먼저
SELECT * FROM users
WHERE age > 25 -- 50만명 (카디널리티 500K)
AND embedding_similarity > 0.8

비용 추정:
- 필터링: 1M × 1ms = 1,000ms
- 임베딩: 500K × 100ms = 50,000ms
- 유사도 검색: 500K × 10ms = 5,000ms
- 합계: 56,000ms

[계획 2] 유사도 검색 먼저
SELECT * FROM users
WHERE embedding_similarity > 0.8 -- ???건 (카디널리티 불명)
AND age > 25

비용 추정:
- 임베딩: 1M × 100ms = 100,000ms ← 모든 사용자!
- 유사도 검색: 1M × 10ms = 10,000ms
- 필터링: ??? × 1ms (결과 크기 불명)
- 합계: 110,000ms + ???

문제: 유사도 검색의 결과 카디널리티를 모르면?
→ 계획 2의 비용을 정확히 추정할 수 없음
→ 옵티마이저가 계획 1 vs. 2를 비교 불가능

문제 해결

SQL과 비교 F

"유사도 0.8 이상인 이미지는 대략 50,000건입니다"

이제 옵티마이저가:
- 계획 1: 500K 임베딩 + 500K 검색 = 55,000ms
- 계획 2: 1M 임베딩 + 1M 검색 + 50K 필터 = 110,050ms
→ 계획 1이 훨씬 나옴!

라고 올바르게 판단할 수 있다.

RAP

4 5

쿼리: 1,000,000명 중
- 임베딩 유사도 > 0.8 → 500,000건 (50% 선택도)
- 나이 > 25 → 500,000건 (50% 선택도)
- 교집합: 약 250,000건

최적 계획: 필터와 유사도 검색을 병렬로 고려
SELECT * FROM users
WHERE age > 25 OR embedding_similarity > 0.8

→ 임베딩과 필터링이 함께 이루어지면 중간 결과 최소화

4 5

쿼리: 1,000,000명 중
- 임베딩 유사도 > 0.9 (매우 유사) → 1,000건 (0.1% 선택도)
- 나이 > 80 (매우 고령) → 100,000건 (10% 선택도)

최적 계획: 더 선택도 높은 것을 먼저 (유사도 검색)
하지만 그 전에 저비용 필터(나이)를 먼저?

1M 임베딩 비용이 매우 크므로
- 나이 필터 먼저: 100K 사람만 임베딩 → $100K \times 100ms = 10K$ 초
- 유사도 검색 필터: 결과 약 1K명

vs.
- 1M 모두 임베딩: $1M \times 100ms = 100K$ 초 (너무 크다)
- 유사도 검색: 1K명

→ 필터 먼저 하는 것이 낫다!

RA

F

95

: 5

3

3

; 5

S4 ~x것

?5

/~개것 ~경과 ~관보

RA m식문변

[5] 비전 논문 (ICDE 2023)
└ E-Operator의 정의 및 동치 규칙 제시
└ "임베딩을 관계형 대수에 통합 가능하다"

↓ (후속 연구)

[6] Context-Enhanced Operators (2023)
└ 비전을 실제 구현으로 변환
└ 텐서 조인으로 100배 성능 향상
└ 인덱스 vs. 스캔 선택도가 중요함을 발견

↓ (파생 연구)

[Exqutor] (암시적)
└ "정확한 카디널리티가 있으면..."
└ 옵티마이저가 최적 계획 선택 가능
└ 카디널리티 추정을 HNSW 인덱스 탐색으로 해결

m식문변

rAt

F

95 복 S군만과고
: 5 복 S군만과고
; 5 복 S군만과고 Wdi n 3경교~과고3mPOi S

95 F
 /PShk3h가PShkx3Vf k0

 M
: 5 F
 / 3 0
 M

; 5 F

?5 F
 M